NS3 - Overview

Dr. S. Amirtharaj
Associate Professor
Mepco Schlenk Engineering College, Sivakasi

Network Simulators

- Network simulators try to model the real world networks. (But can't model all details of the networks.)
- The principal idea is that if a system can be modeled, then features of the model can be changed and the corresponding results can be analyzed.
- The process of model modification is relatively cheap than the complete real implementation, and a wide variety of scenarios can be analyzed at low cost.

Network Simulators (Contd..)

- Anyone can design different network topologies using various types of nodes such as end-hosts, hubs, network bridges, routers, optical link-layer devices, and mobile units.
- It can be used by academic researchers, industrial developers, and Quality Assurance (QA) to design; also to simulate, verify, and analyze the performance of different networks protocols.

Types of Network Simulators

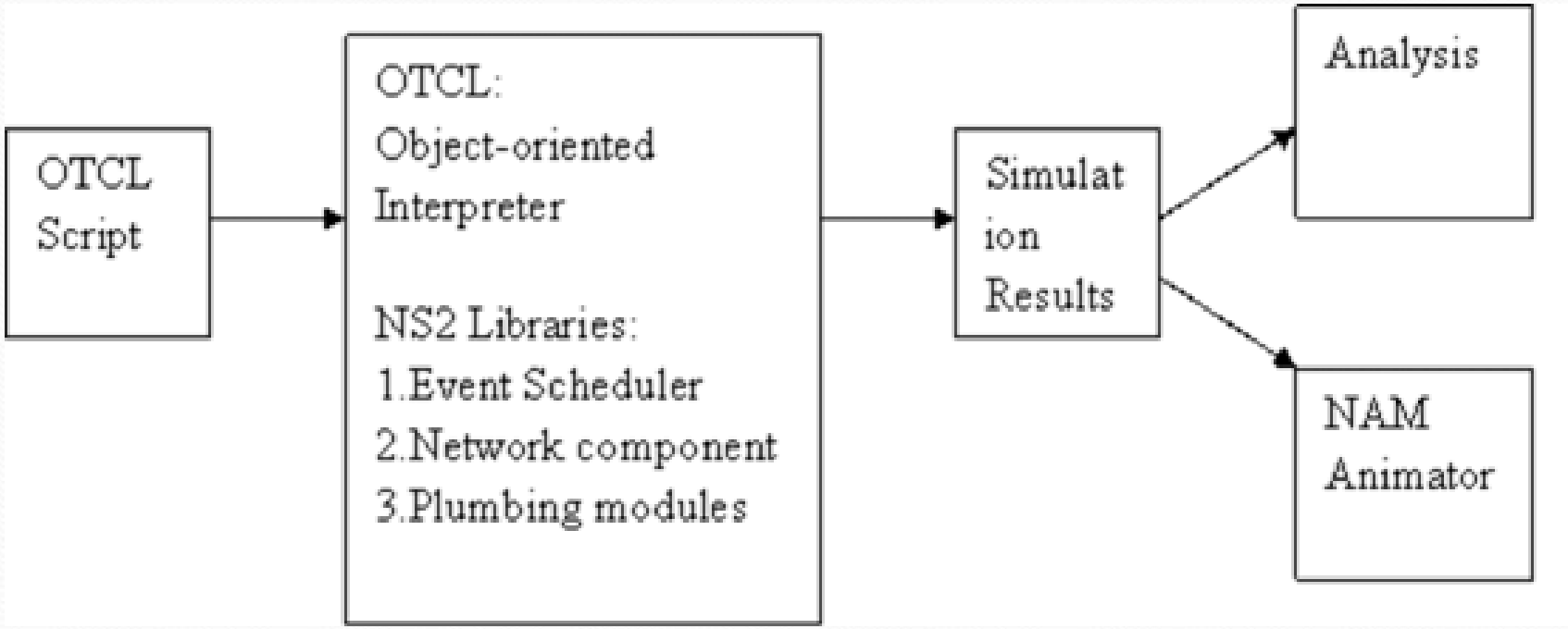
- Commercial and open source simulators
  - OPNET, QualNet
  - NS2, NS3, OMNeT++, SSFNet, J-Sim
- Graphical vs Text-Based Simulator
  - Graphical simulators allows user to visualize the working of the networking environment. Eg. QualNet
  - Text-based simulators provide a less visual interface, but may allow more advanced forms of customization. Eg. NS2 & NS3

NS2

- One of the most popular open source network simulators.
- Discrete event simulator (process of coding the behavior of complex system as an ordered sequence of well-defined events) targeted at networking research.
- The first version of NS was developed in 1989.
- It uses C++ and OTcl (Tcl script language with Object-oriented concepts)
- C++ is efficient to implement a design but it is not easy to modify and assemble different components and to change different parameters without a visual and easy-to-use descriptive language.

NS2 Contd….

- C++ is used to implement the detailed protocol and OTcl is used for users to control the simulation scenario and schedule the events
- Simplified user's view of NS2,



Other Network Simulators

Quite a few actually, for ex:

NS-2

J-Sim

TOSSIM

GloMoSim

Agent J

SHARPE

OPNET

ns-3

Netsim

Qual Net

SPNP

OMNET++

INSANE

TrSim

Query Cycle

Malsie

Neurogrid

We will focus on ns-3

Introduction to NS3

- ns-3 simulator is a discrete-event network simulator targeted primarily for research and educational use.
- ns-3 is not a backwards-compatible extension of ns-2; it is a new simulator (The two simulators are both written in C++ but ns-3 is a new simulator that does not support the ns-2 APIs).
- ns-3 is open-source, and the project strives to maintain an open environment for researchers to contribute and share their software / code.

Why NS3…???

- To perform studies that are more difficult or not possible to perform with real systems.
- To learn about how networks work.
- Focuses on modeling how Internet protocols and networks work.
- It is possible to model non-Internet-based systems in ns3.

Features of NS3

- ns-3 is designed as a set of libraries that can be combined together and also with other external software libraries.
- New animators and visualizers are available and under current development of NS3
- ns-3 generates **pcap** (packet capture which consists of an API for capturing network traffic) packet trace files, which can be used to analyze the network design
- ns-2 models can be ported to ns-3, if they have been implemented in C++.

NS2 vs. NS3

NS-2	NS-3
NS2 libraries are written in C++	NS3 libraries are entirely written in both C++ and Python.
Simulation and events can be scheduled by OTcl.	Simulation and events can be scheduled by either C++ or Python codes.
Simulation can be visualized by Network Animator (netsim)	Simulation can be visualized by more Network Animator tools(netanim)
Need to write a program to trace the packets from the trace file (.tr)	ns-3 generates .pcap packet trace file / .xml trace file to analyze the network design

SETTING UP NS3 ENVIRONMENT

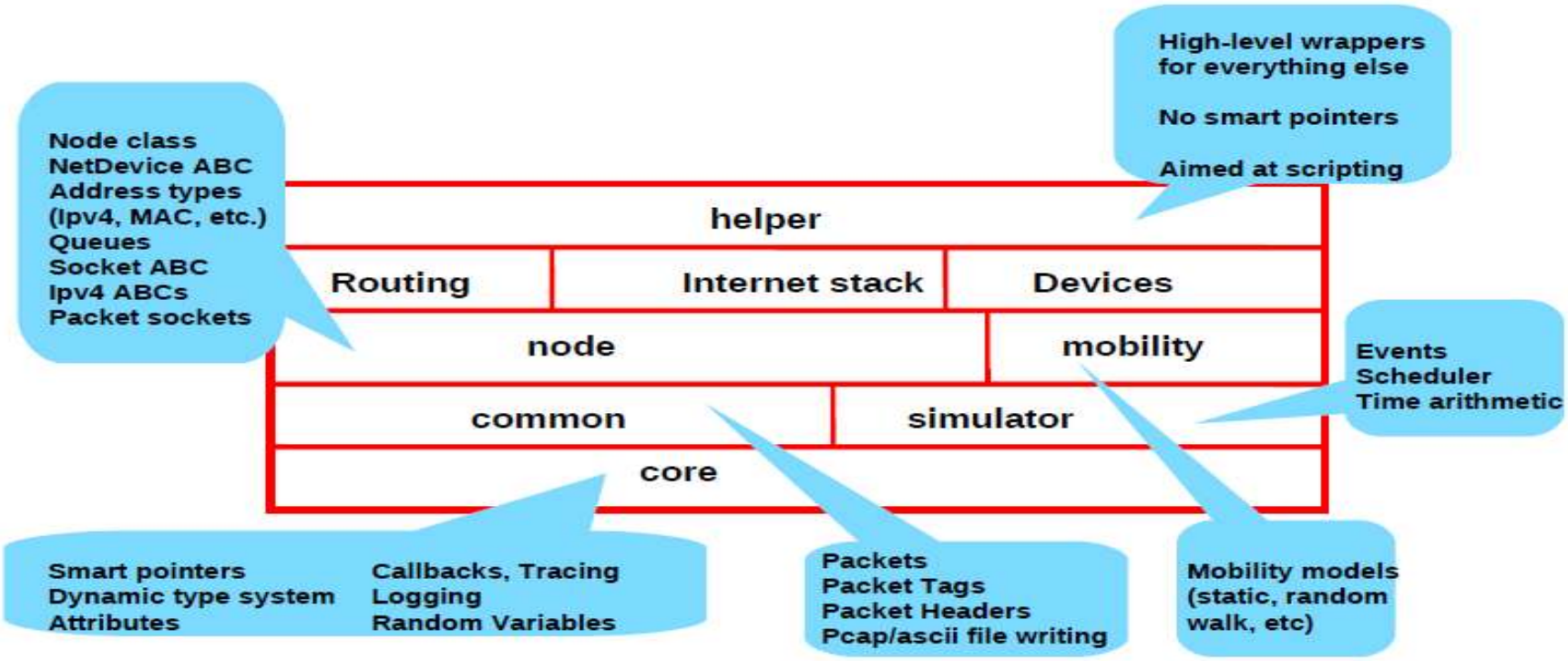
- Mercurial : Source Code Management System
- Waf: Python based Build system
- Environment: Linux or Linux based

- Prerequisites:
  - gcc/g++ 3.4 or greater
  - Python 2.4 or greater
  - More packages to support different ns-3 options
- Downloading ns-3
- Building ns-3
- Testing ns-3
- Running a script

Installation of NS3

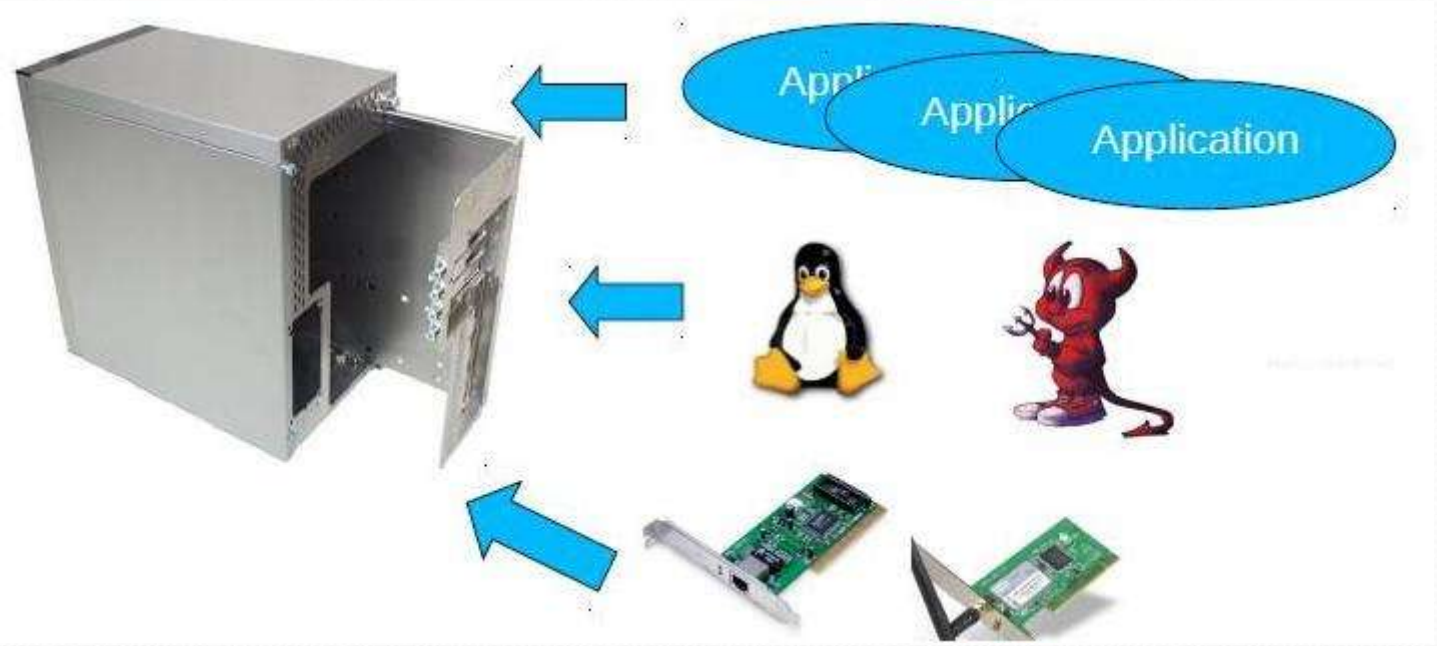
CONCEPTUAL OVERVIEW

A software organization view



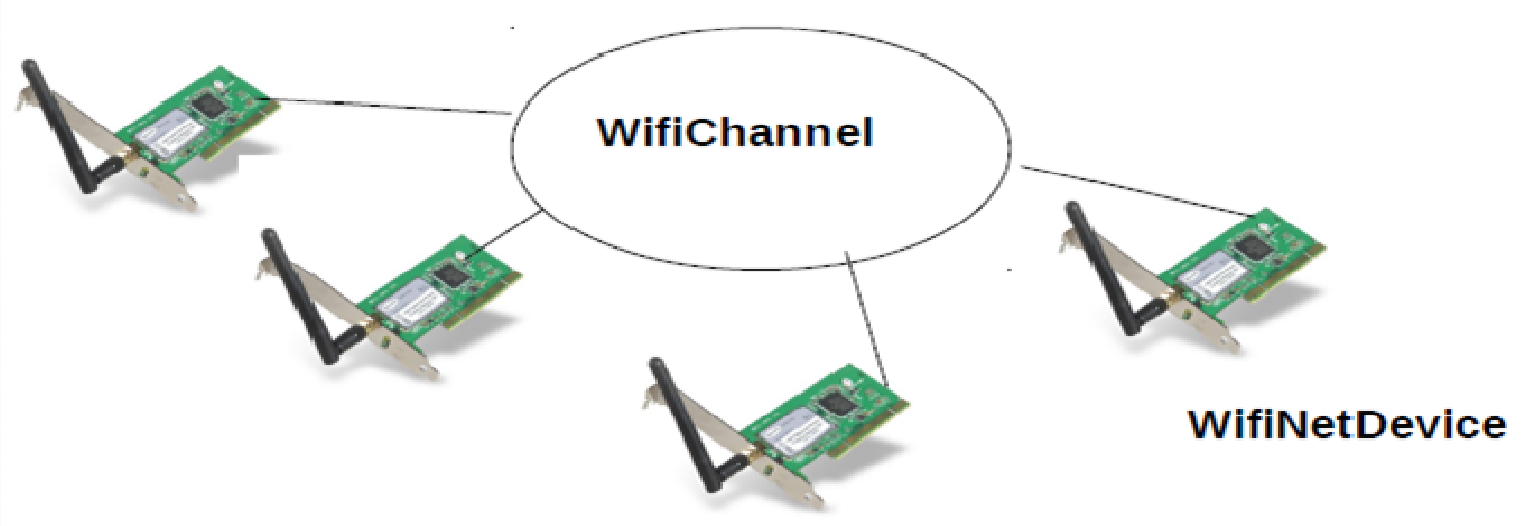
NODE

- Basic computing device abstraction is called the node. This abstraction is represented in C++ by the class Node.
- i.e., Node is a container which has the functionalities such as applications, protocol stacks, NICs, etc.



NET DEVICES

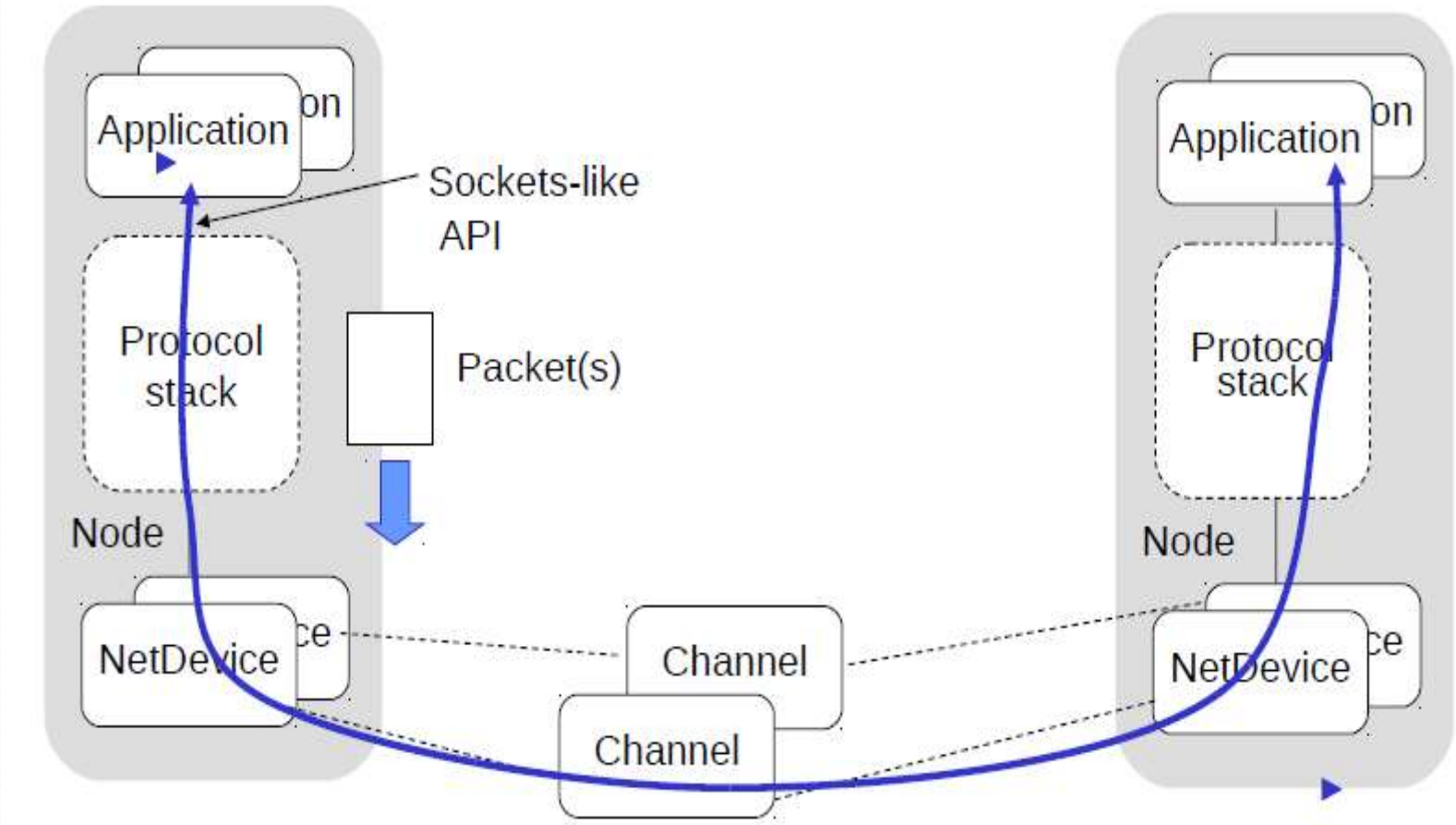
- A net device is installed in a Node in order to enable the Node to communicate with other Nodes in the simulation via Channels.
- NetDevice class provides methods for managing connections to Node and Channel objects.



Application & Channel

- **Application** - ns-3 applications run on Nodes to drive simulations in the simulated world. i.e, user program is the application which generates some activity to be simulated.
- **Channel** – connects a Node to an object representing a communication channel. Channel class provides methods for managing communication subnetwork objects and connecting nodes to them.

BASIC MODEL


Application
Sockets-like
API
Protocol
stack
Packet(s)
Node
NetDevice
Channel
Channel
Application
Protocol
stack
Node
NetDevice

HELPER MODELS

- TopologyHelper: It does the common tasks in ns3 such as connecting NetDevices to Nodes, NetDevices to Channels, assigning IP addresses, etc.
- InternetStackHelper,
- WifiHelper,
- MobilityHelper,
- OlsrHelper,
- …… Each model provides a helper class

OTHER MODELS

- Devices
  - WiFi, WiMAX, CSMA, Point-to-point, Bridge
- Error models and queues
- Applications
  - echo servers, traffic generator
- Mobility models
- Packet routing
  - OLSR, AODV, Static, Nix-Vector, Global (link state)

Structure of an ns-3 program

```
int main (int argc, char *argv[])
{

  // Set default attribute values

  // Parse command-line arguments

  // Configure the topology; nodes, channels, devices, mobility

  // Add (Internet) stack to nodes

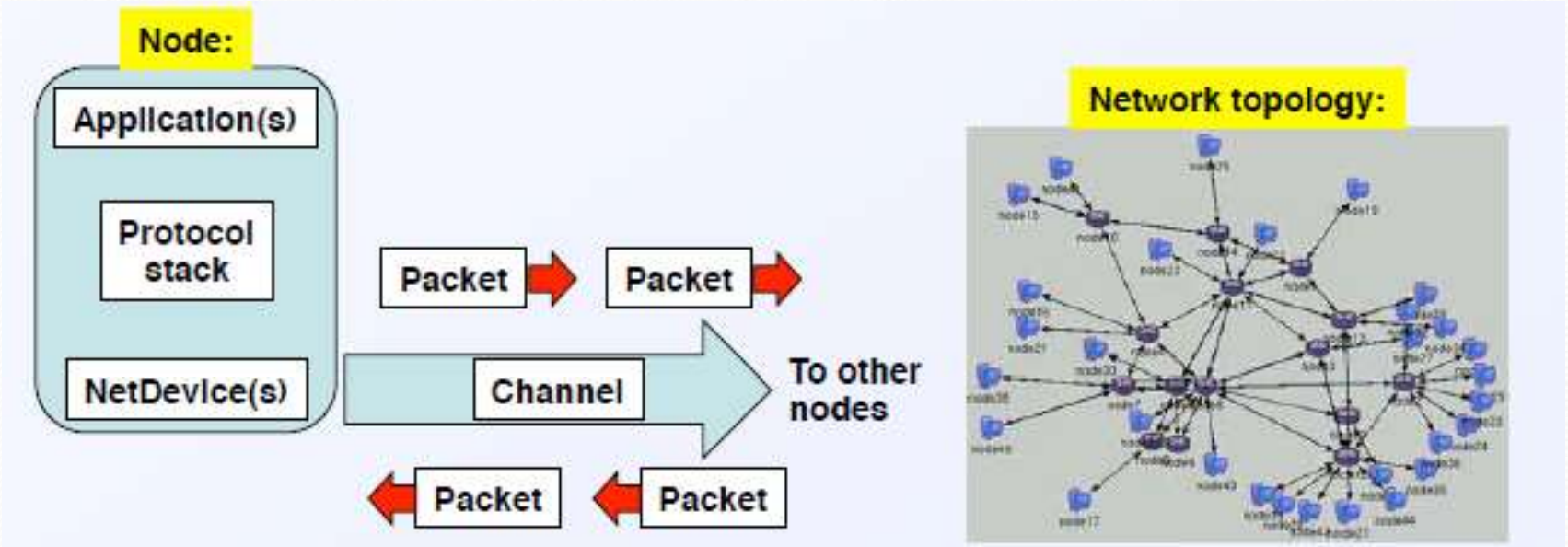
  // Configure IP addressing and routing

  // Add and configure applications

  // Configure tracing

  // Run simulation
}
```

Simulation in NS3



- Nodes: hosts, routers, servers,…
- Applications: generate and consume traffic in network
- Protocols: broker connections, access, addressing, routing,…
- NetDevices: e.g., ethernet & wireless cards
- Channels: transmission medium (cable, EM waves,…)
- Packets: make up network traffic

Basic Classes in NS3

- Command Line Arguments
- Logging
- Container Classes
  - NodeContainer,
  - NetDeviceContainer,
  - ApplicationContainer,
  - Ipv4InterfaceContainer, etc..
- Helper Classes
  - PointToPointHelper
  - CsmaHelper
  - IPv4AddressHelper
  - InternetStackHelper
  - MobilityHelper
  - WifiHelper
  - Ipv4GlobalRoutingHelper, etc..
- Animation & Tracing

Command Line Arguments

```
int main (int argc, char *argv[])
{ ...
   CommandLine cmd; uint32_t nPackets = 1;
   cmd.AddValue("nPacketsArg", "Number of packets",
   nPackets);   //to set local variable from command line
   cmd.Parse (argc, argv);  //Parse the command line arguments
... }
```

- ./waf --run “scratch/first1 --ns3::PointToPointNetDevice::DataRate=5Mbps”
- $ ./waf --run “scratch/first1 --nPacketsArg = 3”

Logging

- NS3 log module is used to get useful information as messages out from the simulation to execution environment.
- Seven levels of log messages in NS3 are as follows:
  - LOG_ERROR – log error messages
  - LOG_WARN – log warning messages
  - LOG_DEBUG – debugging messages
  - LOG_INFO – informational messages with ERROR, WARN, DEBUG messages
  - LOG_FUNCTION – message for function call
  - LOG_LOGIC – messages for logical flow within a function
  - LOG_ALL – log everything given above

Logging (Cont..)

- Log component can be created for our program by,

```
NS_LOG_COMPONENT_DEFINE  ("FirstScriptExample");
```

- To see any logs in the output, we have to enable that log component by,

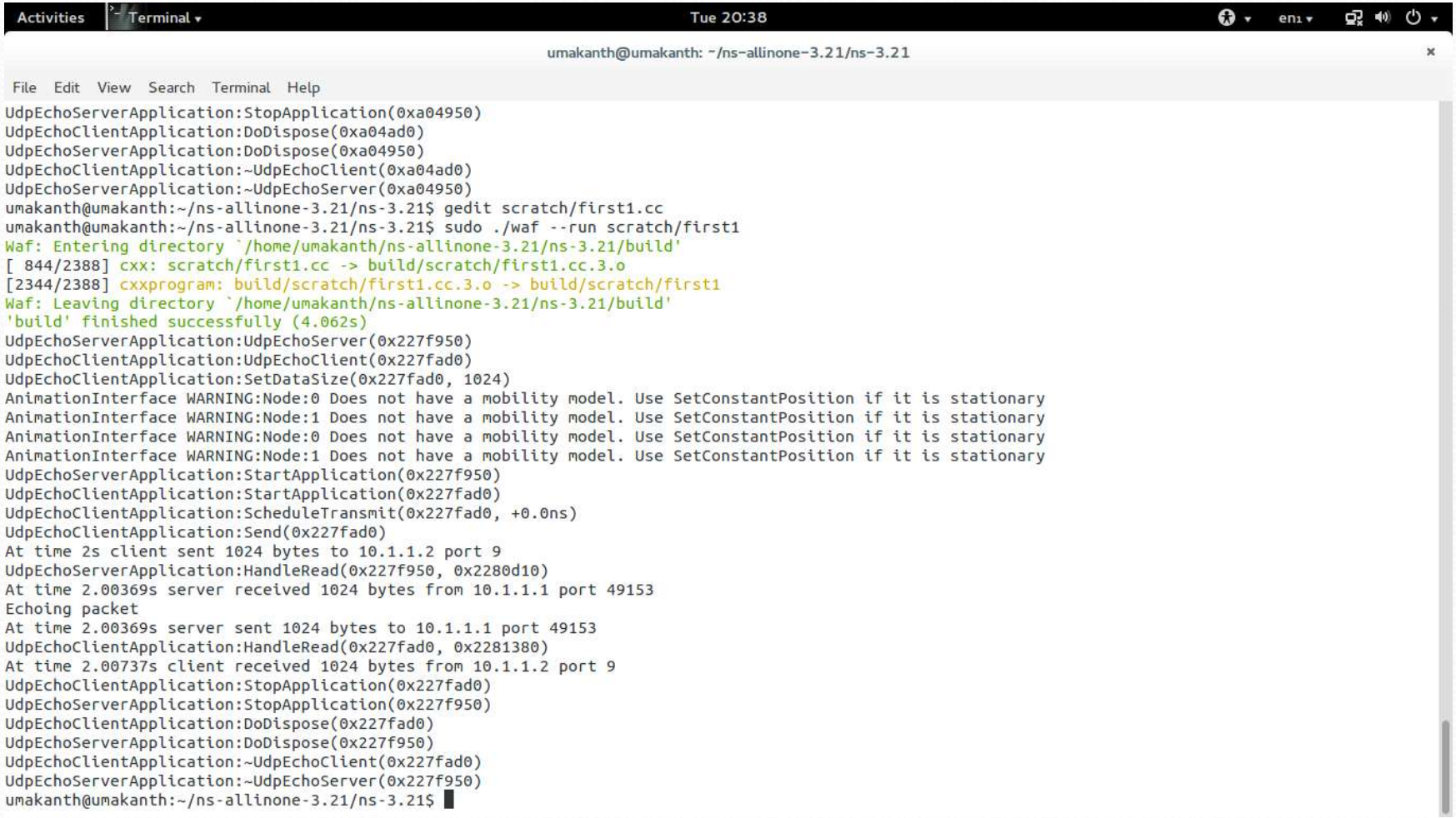
```
LogComponentEnable ("FirstScriptExample", LOG_LEVEL_INFO);
```

- User defined logs can be created by,

```
NS_LOG_INFO ("Creating Topology");
```

which will be displayed in the output screen while executing our script.

By using LOG_ALL



Container Classes: NodeContainer

- NodeContainer is used to create, manage and access any Node objects that are created in order to run the simulation.
- NodeContainers hold the multiple Ptr<Node> which are used to refer to the nodes.
- Attributes in NodeContainer:
  - Iterator: index value which refers the node
  - m_nodes: Number of smart pointers for the nodes
- Member Functions in NodeContainer:
  - Add (NodeContainer nc) : Append the contents of another NodeContainer to the end of this container.
  - Add (Ptr< Node > node) : Append a node to the end of the container
  - Begin (void) const : Get the Iterator for the First node in the container
  - Create (uint32_t *n*) : Create n Nodes and appends them at the end
  - End (void) const : Get the Iterator for the Last node in the container
  - Get (uint32_t *i*) : Get the Ptr<Node> stored in the container of index I
  - GetN (void) const : Get the number of nodes stored in this container

Sample program

p2p.cc program: a script that will create a simple point-to-point link between two nodes and echo a single packet between the nodes.

Module Includes

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h“
```

Ns3 Namespace:

```
using namespace ns3;
```

It is used to access all the functionalities in ns3; if not so, it can be accessed by ns3::function()

Logging: LOG functionality is a macro which allow developers to send information to the std::clog output stream

```
NS_LOG_COMPONENT_DEFINE  ("FirstScriptExample");
```

Sample Program (Contd…)

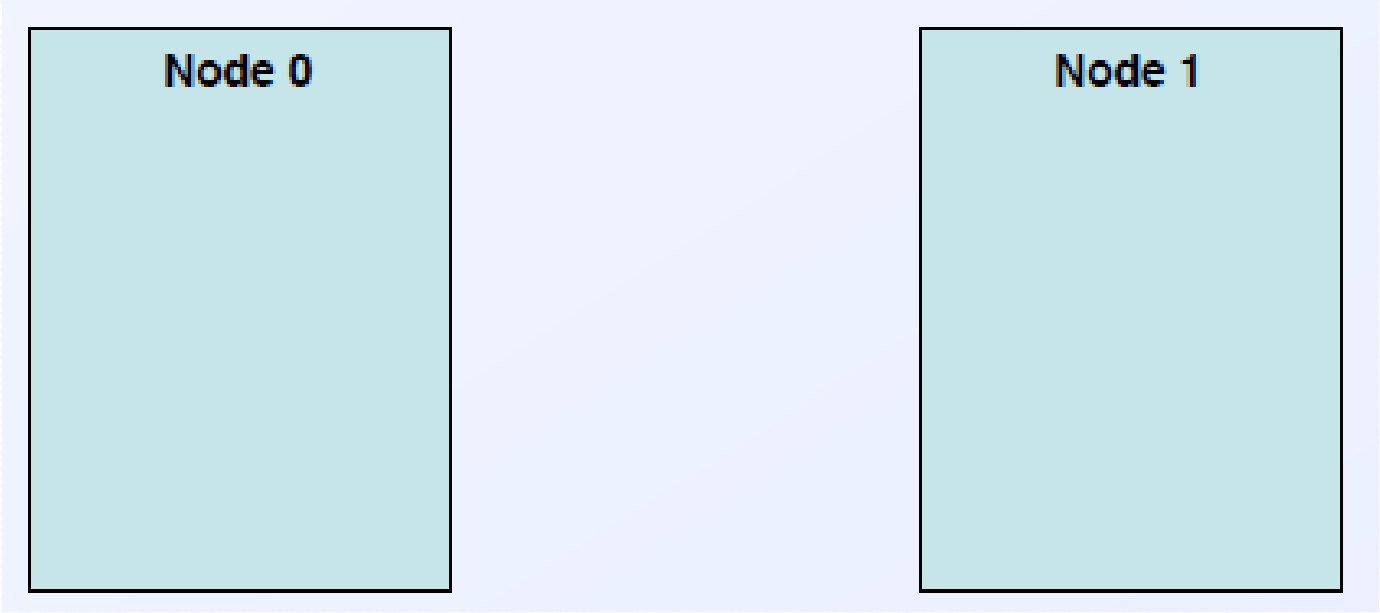
```
int main (int argc, char *argv[])
{
//set the time resolution as 1 NanoSecond (Default)
Time::SetResolution (Time::NS);
/*to enable two logging components that are built into the Echo Client and
Echo Server applications*/
//logging messages are disabled by default, have to be enabled by
LogComponentEnable("UdpEchoClientApplication", LOG_LEVEL_INFO);
LogComponentEnable("UdpEchoServerApplication", LOG_LEVEL_INFO);
```

Sample Program (Contd…)

//The NodeContainer topology helper is used to create, manage and access any Node objects that are created in order to run a simulation

```
NodeContainer   nodes;

nodes.Create (2);   //creates 2 nodes on the NodeContainer object
```



Sample Program (Contd...)

```
PointToPointHelper    pointToPoint; // Used to create P2P Network easily
/* Assigns the DataRate as 5Mbps to be used when it creates
PointToPointNetDevice object. */
pointToPoint.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
//Assign the Delay attribute which is associated with PointToPointChannel
pointToPoint.SetChannelAttribute ("Delay", StringValue ("2ms"));
```



Sample Program (Contd...)

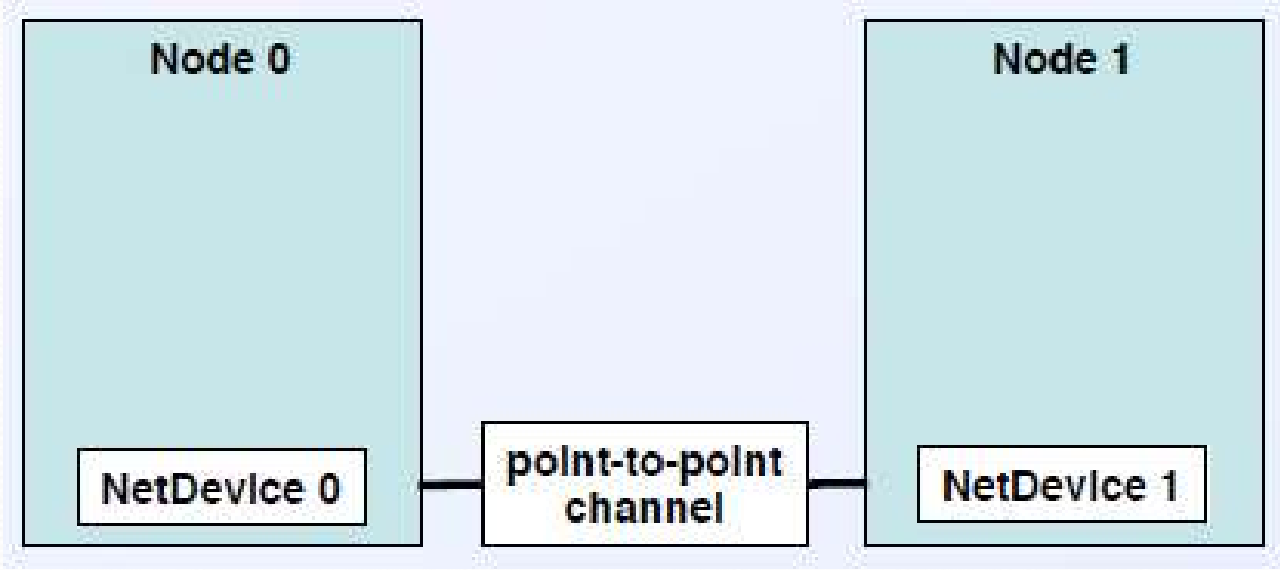
//create Container object to hold all the NetDevice objects.

NetDeviceContainer devices;

// To configure the NetDevices and Channel.

devices = pointToPoint.Install (nodes);

/* Install method creates the PointToPointChannel and 2 PointToPointNetDevices are attached on it. Now both devices are configured to transmit data at five megabits per second over the channel which has a two millisecond transmission delay. */

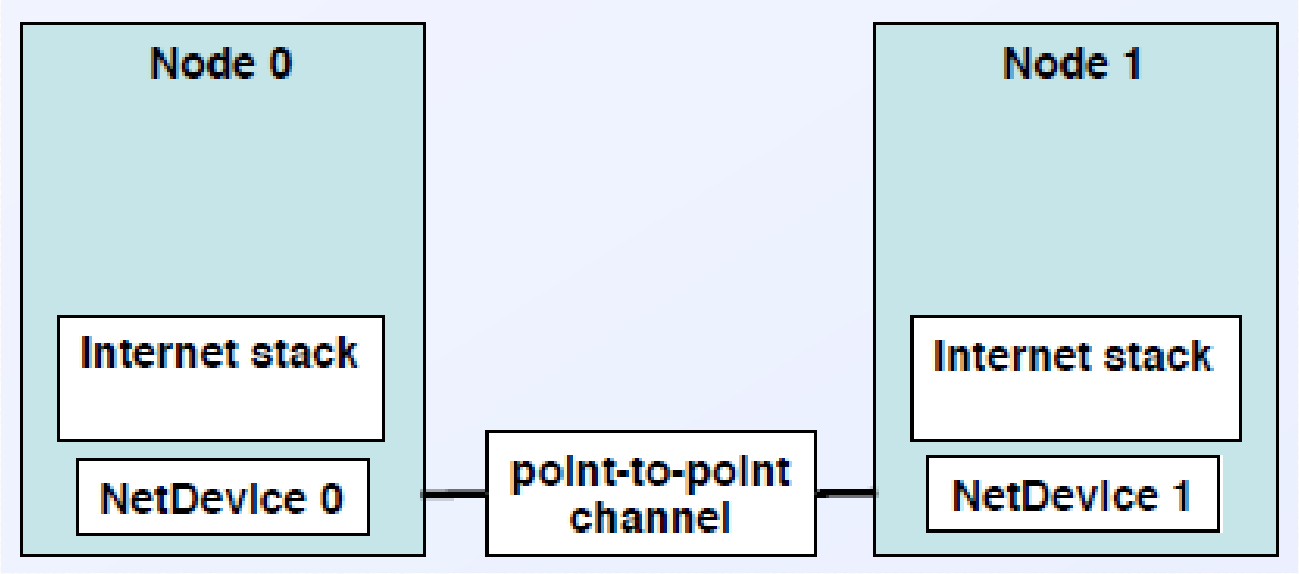


Sample Program (Contd...)

//To install the Internet Stack (TCP, UDP, IP, etc.) on each nodes

InternetStackHelper stack;

stack.Install (nodes);



Sample Program (Contd...)

/* Ipv4AddressHelper is used to generate simple IPv4 Addresses for each nodes */

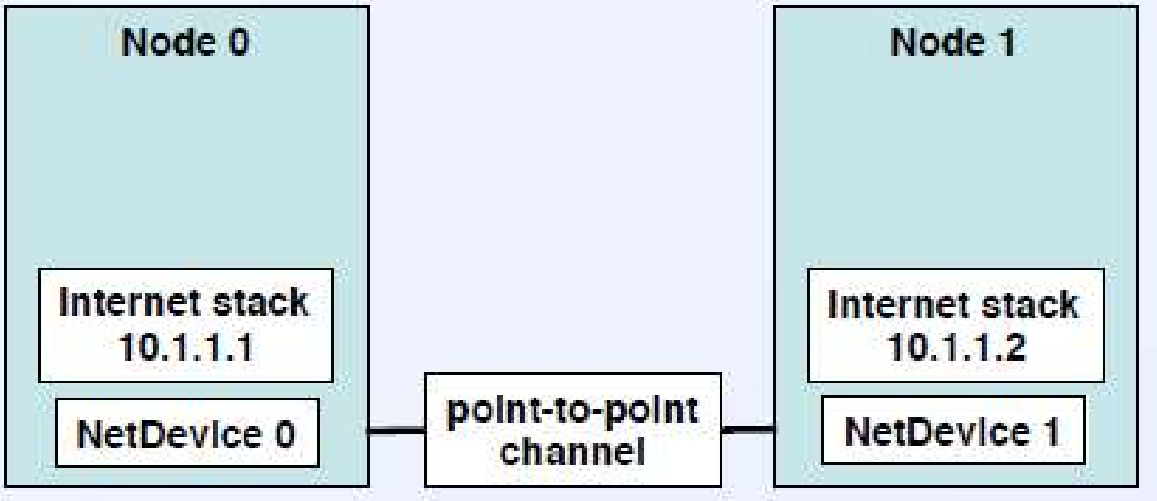
```
Ipv4AddressHelper   address;
```

/* To allocate the IP addresses from 10.1.1.0 using the mask 255.255.255.0. ie., starts from 10.1.1.1 to 10.1.1.254 */

```
address.SetBase ("10.1.1.0", "255.255.255.0");
```

// Associate IP address and NetDevice using Ipv4Interface object.

```
Ipv4InterfaceContainer   interfaces = address.Assign (devices);
```



Sample Program (Contd…)

- Finished Building PointToPoint network topology; with Network Stack installed and IP addresses assigned as well.
- Now we have to create an echo application to transfer the packet between the nodes.
- So we can use the helper classes such as UdpEchoServerHelper and UdpEchoClientHelper to set up the client & server applications in the created nodes.

Sample Program (Contd...)

/* Create Helper object to create the application "UDP Echo Server"

(1500 – Port number) */

UdpEchoServerHelper echoServer (9);

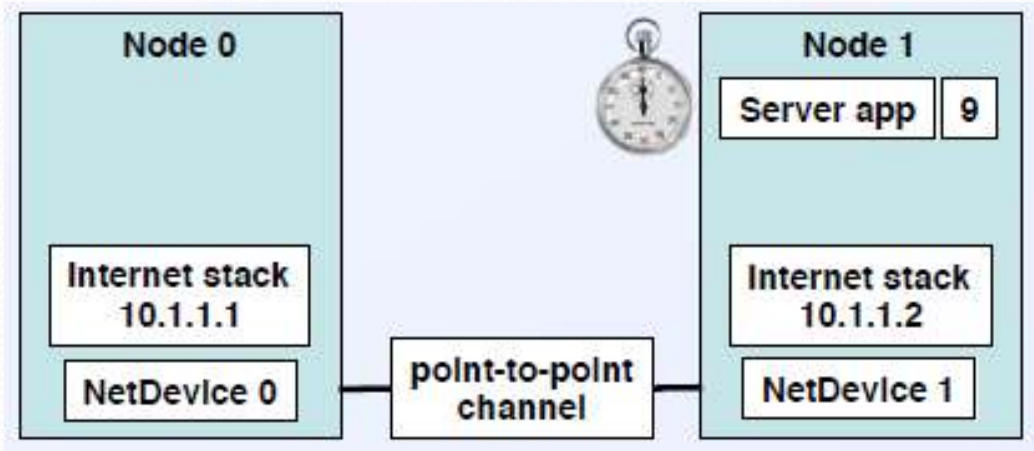
// Install() instantiate the Echo Server Application and attached to a node
// nodes.Get (1) returns a smart pointer to a node object — Ptr<Node>

ApplicationContainer serverApps = echoServer.Install (nodes.Get (1));

// Make echo server application to Start at 1 sec and to Stop at 10 sec in the simulation.

serverApps.Start (Seconds (1.0));

serverApps.Stop (Seconds (10.0));



Sample Program (Contd…)

- Server application is created to wait for input UDP packets and sends them back to the original sender.

- Next step is to Create a client application which sends the UDP packet and waits for an echo of the packet.

Sample Program Contd...

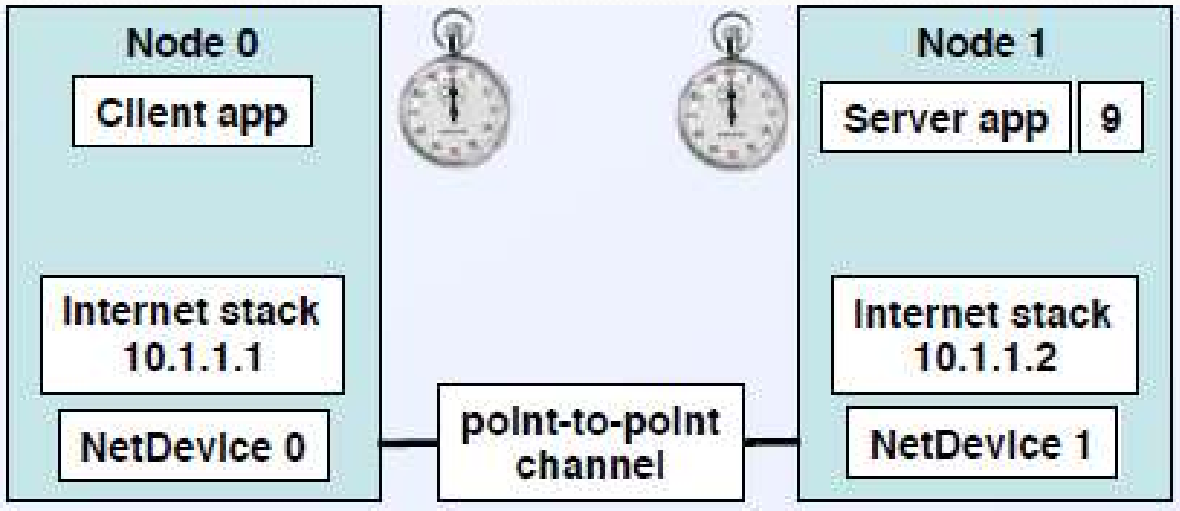
```
// Echo client application creation
// IP address and port number of the remote UDP echo server
UdpEchoClientHelper   echoClient (interfaces.GetAddress (1), 9);
// Maximum number of packets, interval between the packets and
packet size
echoClient.SetAttribute ("MaxPackets", UintegerValue (1));
echoClient.SetAttribute ("Interval", TimeValue (Seconds (1.0)));
echoClient.SetAttribute ("PacketSize", UintegerValue (1024));
```

Sample Program Contd…

// Like server application, we need to start and stop the Echo Client

// Install() instantiate the Echo Server Application and attached to a node

```
ApplicationContainer   clientApps = echoClient.Install (nodes.Get (0));
clientApps.Start (Seconds (2.0));
clientApps.Stop (Seconds (10.0));
```



Sample Program Contd…

// Run the application; to execute the list of scheduled events (Start event at 1.0, 2.0 seconds and Stop event at 10.0 seconds)

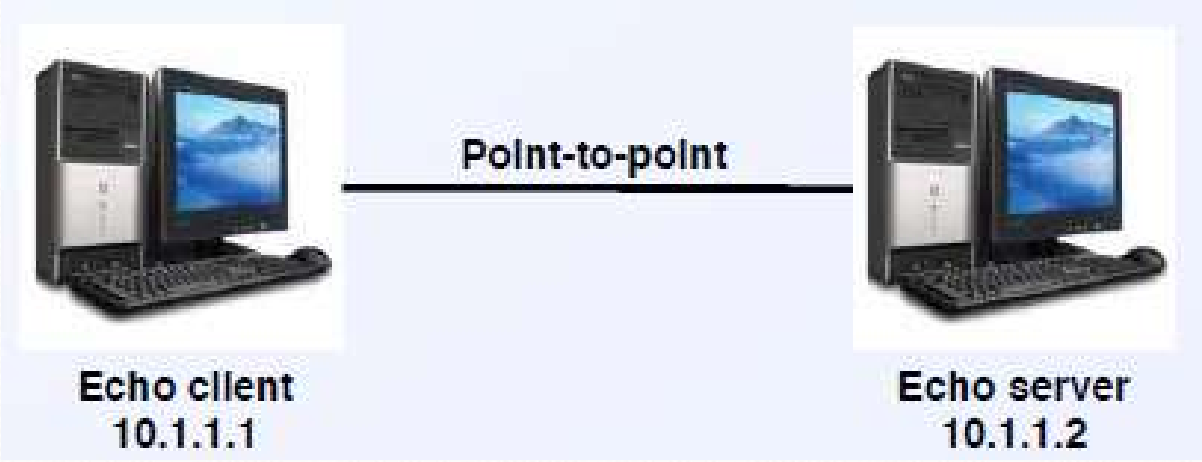
Simulator::Run ();

// To destroy all the objects that were created

Simulator::Destroy ();

return 0;

}



Building & Running the Program

- Save the program in ns-allinone2.31/ns3/script directory

- Build the script using Waf

  $ ./waf

- Run the program

  $ ./waf --run scratch/p2p.cc

Sample output

Waf: Entering directory ‘/home/craigdo/repos/ns-3-allinone/ns-3-dev/build’
Waf: Leaving directory ‘/home/craigdo/repos/ns-3-allinone/ns-3-dev/build’
’build’ finished successfully (0.418s)
Sent 1024 bytes to 10.1.1.2
Received 1024 bytes from 10.1.1.1
Sent 1024 bytes to 10.1.1.1
Received 1024 bytes from 10.1.1.2

Hands-on….

1. **Create a simple topology of two nodes (Node1, Node2) separated by a point-to-point link.**

2. **Setup a UdpClient on one Node1 and a UdpServer on Node2. Let it be of a fixed data rate Rate1.**

3. **Now add another client application to Node1 and a server instance to Node2.**

4. **What do you need to configure to ensure that there is no conflict?**

RESOURCES

- http://www.nsnam.org provides access to basic information about the ns-3 system.
- Detailed documentation is available at http://www.nsnam.org/documentation/
- Wiki that complements the main ns-3 web site can be found at http://www.nsnam.org/wiki/
- The source code may be found at http://code.nsnam.org/ and current development tree is in the repository named ns-3-dev

THANK YOU